FOUNDATIONS OF AI / PROJECT DELIVERABLE

Handwork

Product Requirements Document

A native terminal harness for reviewable AI coding work.

Product owner / author	Document snapshot
Connor Love (repository attribution)	October 8, 2026
Status: review draft	Native source/package: 0.1.0

Executive summary

Handwork connects a model to a local coding project through its own agent loop, terminal interface, file and command tools, permission checks, and saved conversations. Developers can choose a native model route, use separate official runtime adapters, or embed an agent through libhandwork.

The product should turn a concrete coding request into a focused, inspectable change with actual verification evidence. Its proposed quality baseline centers on clear setup, explicit authority, responsive execution, preservation of developer work, and honest recovery after interruption.

This document grounds the product in the inspected repository, defines testable requirements, and identifies decisions that need owner approval. It does not invent user research, release readiness, speed advantages, or provider authorization.

Contents

01 Document purpose and evidence / 3
02 Problem and product opportunity / 4
03 Users and jobs to be done / 5
04 Goals, scope, and non-goals / 6
05 Product model and execution boundaries / 7
06 Core journey: a verified repository repair / 8
07 Alternate journeys and failure recovery / 9
08 Task and tool state contracts / 10
09 Requirements: installation and first use / 11
10 Requirements: providers and model capabilities / 12
11 Requirements: project context and workspace scope / 13
12 Requirements: file tools and review / 14
13 Requirements: shell and managed processes / 15
14 Requirements: permissions and user control / 16
15 Requirements: sessions, resume, and undo / 17
16 Requirements: model-led subagents / 18
17 Requirements: skills, MCP, and images / 19
18 Requirements: scripting, GitHub, and usage / 20
19 Requirements: SDK and ACP contracts / 21
20 Requirements: terminal UX and accessibility / 22
21 Security, privacy, and data lifecycle / 23
22 Nonfunctional requirements and performance / 24
23 Desired impact and success measures / 25
24 Proposed measurement and event contracts / 26
25 End-to-end acceptance scenarios / 27
26 Verification plan and traceability / 28
27 Delivery sequence and dependencies / 29
28 Constraints, tradeoffs, and risk register / 30
29 Open decisions and proposed ownership / 31
30 Release acceptance and definition of done / 32
31 Sources and evidence limits / 33
32 Glossary and assignment coverage / 35

Linked contents and PDF bookmarks support navigation. Requirement references connect to the source register; test IDs connect to the verification plan.

01 / SPECIFICATION

Document purpose and evidence

This PRD specifies Handwork as an existing product and defines a proposed quality baseline for its next qualified release. It is prepared for the Foundations of AI project deliverable and for product and engineering review. It is a specification, not a release announcement or an implementation audit.

How to read the requirements

Label	Meaning
Baseline	Behavior documented in the repository or visible in inspected source. Acceptance criteria specify what a release must demonstrate; they do not mean that behavior was executed or passed during this review.
Proposed	An explicit recommendation for future product behavior, measurement, or release process. It requires owner approval before becoming a delivery commitment.
Hypothesis	A user need or expected impact inferred from the product and its documented direction. It is not a finding from interviews or production analytics.
Open decision	A material choice or fact the available repository cannot resolve. The document states the consequence and proposed decision owner.

Evidence snapshot

Repository: loveconnor/handwork. Local checkout inspected on October 8, 2026. Commit: 0752c714101ebfefc3e58dc0e0e7db36c35ee8cd. Source references [S01]-[S18] appear at the end. The screenshot supplies the assignment rubric: problem, users, core journey, requirements, desired impact, and constraints.

The native entry point and package declare 0.1.0; the README and Codex adapter client metadata still mention 0.0.9. The SDK manifest declares 0.0.0. Registry publication status was not checked. Historical benchmark reports were not inspected for this PRD; the referenced provider-terms audit file is absent from this checkout. Neither is treated as verified release evidence. [S01, S02, S03, S11, S12]

No live model calls, external publishing, credentials inspection, product test runs, user interviews, billing validation, or fresh benchmark measurements were performed. This document cannot establish release readiness or provider-policy compliance.

02 / SPECIFICATION

Problem and product opportunity

Problem statement

A developer who delegates coding work needs a model to understand the repository, make a focused change, execute the right checks, and leave an inspectable result. The work also needs an execution environment: file tools, command management, instructions, permission decisions, saved context, and a usable interface. A chat response alone cannot supply those guarantees.

Handwork supplies that environment through a native terminal harness. Its opportunity is to make local interaction and tool execution economical and predictable while preserving developer control across different model providers. This is a product hypothesis supported by the repository direction, not proof that developers prefer Handwork. [S01, S04]

Specific problems to solve

Moment	User problem	Required outcome
Setup	API credentials, subscription runtime access, endpoint selection, and billing can be confused.	The user knows which engine and account route will run the task before sending code.
Execution	The model can make an incorrect edit, run an unsafe command, or misreport a tool result.	Actions pass the appropriate controls and leave reviewable evidence.
Long work	Large transcripts, command output, search indexes, and diffs can obstruct typing and review.	The terminal stays operable under representative stress.
Interruption	A cancelled command or interrupted write may have already changed the workspace.	The user can distinguish known completion, partial effects, and unknown state.
Integration	An embedded agent can silently assume CLI permissions or filesystem authority.	The host can understand and own each integration boundary.

Why the project is specific

Handwork combines a Zig runtime and terminal with a native agent loop, provider adapters, and a separate JavaScript embedding surface. Fast startup, low local memory use, and responsive terminal interaction are design goals. They are not evidence of universally better code or a blanket speed advantage over other agents. [S01, S03]

03 / SPECIFICATION

Users and jobs to be done

The following segments are inferred from existing entry points and documentation. They are working audience hypotheses until validated with representative users. No demographic personas, customer quotes, or adoption figures are invented.

Segment	Context and job	Success and likely concern
Primary: terminal developer	Works in a local repository; uses Git, a shell, and project tests. Delegates a bounded repair, explanation, or feature task.	Gets a focused diff with actual check results. Needs to preserve existing work and understand the agent's authority.
Secondary: provider-flexible developer	Chooses among API providers, local model servers, and official runtime routes.	Can select a compatible route without credential confusion or a hidden fallback. Needs clear capability and data-routing limits.
Secondary: automation author	Invokes one request from scripts and parses output. May run without a terminal.	Receives parseable results, explicit failure, and predictable permission behavior. Needs to distinguish partial output from a completed response.
Secondary: SDK or editor integrator	Embeds an agent or terminal in a JavaScript host, or connects an ACP client.	Owns authentication, tool effects, persistence, cancellation, and cleanup without undocumented CLI dependencies.
Supporting: project maintainer	Maintains instructions, skills, MCP configuration, release evidence, and product behavior.	Can explain scope, reproduce defects, qualify artifacts, and remove unsafe or unsupported integrations.

Primary job

When I have a concrete coding task in a repository, I want to delegate inspection, focused editing, and verification from my terminal so I can review a useful change without losing control of my files, credentials, or running processes.

Research needed before broad positioning

Observe first-time setup, one repair task, one interrupted task, and diff review with terminal developers. Include keyboard-only and assistive-technology users. Ask what they understood about permission mode, data destination, billing, and resume. Recruitment, sample size, and research owner remain open; the study must report actual participants and limits.

04 / SPECIFICATION

Goals, scope, and non-goals

Product goals

G1: A developer can reach a first useful, reviewable task with a clearly selected engine and model. G2: Native actions follow effective permissions and workspace scope. G3: Long work remains interruptible and reviewable. G4: Saved work can be resumed with an honest account of what persistence restores. G5: Integrators can embed the runtime with explicit host ownership.

Scope of this PRD

Area	Included
Native experience	Interactive terminal; one-shot ask; file and command tools; instructions; provider/model choice; permissions; context budgets; sessions; tracked-file undo.
Extensions	Model-led subagents, skills, MCP, additional workspace directories, image input where supported, GitHub draft and publish boundaries.
Integration	Separate official runtime adapters, ACP endpoint, libhandwork agent and terminal surfaces, host-owned tools and persistence.
Product quality	Errors and recovery, terminal accessibility, data boundaries, local usage semantics, test evidence, performance qualification, packaging and release consistency.

Non-goals for the proposed baseline

No new hosted team dashboard, cloud agent scheduler, mobile app, marketplace, billing service, or bundled editor extension. No promise of supported CLI packages beyond qualified macOS arm64 distribution. No automatic transfer of another tool's settings or sessions. No guarantee of model correctness, universal local-only inference, or operating-system sandboxing.

Broad cross-platform CLI rollout and a hosted release service remain later work until their support, signing, update, rollback, and maintenance costs are approved. The SDK has a different platform matrix and must be specified separately. [S01, S02, S11]

Priority convention

P0: mandatory for the proposed baseline because it protects the core task, truthfulness, or user control. P1: important follow-on quality or integration work. P2: later exploration. These are document priorities; they do not represent an approved staffing plan or release date.

05 / SPECIFICATION

Product model and execution boundaries

Three surfaces with different owners

Surface	Execution and authority	Persistence and limits
Native Handwork	The Zig agent loop uses Handwork tools, provider transports, workspace scope, and native permission policy.	Native saved sessions and history. Source and npm distribution support must be distinguished.
Official runtime adapter	The provider engine owns tools, configuration, sessions, and sandbox behavior. Handwork mediates its adapter interface.	Provider-owned session identifiers. Native permission settings do not configure that engine.
libhandwork / ACP	The JavaScript agent host owns credentials and tool effects; the terminal harness is a separate API. ACP exposes native operation over stdio.	SDK checkpoints are host-stored and require integrations to be supplied again. ACP is an endpoint, not an editor extension.

Core objects

Workspace: primary directory plus active additional roots. **Session:** saved conversation and related execution records. **Turn:** one user request through completion, interruption, or failure. **Tool action:** structured attempt with target, arguments, policy decision, execution, and result. **Managed command:** owned process with observation and stop operations.

Provider/model selection: explicit endpoint, model ID, credentials source, and capabilities. **Permission grant:** scoped authorization with a policy source. **Child conversation:** bounded delegated work with inherited or explicit creation-time configuration. **Checkpoint:** opaque SDK conversation state, not a filesystem backup. [S04-S12]

Trust boundaries

User intent and authorized configuration must remain distinct from repository text, command output, fetched content, tool results, and model proposals. Handwork's checks govern its own native dispatch path. A shell process, a remote MCP service, an official runtime, or a host tool may have capabilities that require separate containment and policy.

06 / SPECIFICATION

Core journey: a verified repository repair

Representative scenario: a terminal developer asks Handwork to fix a failing parser test, keep the patch focused, run the parser tests, and explain the result. This is an illustrative task, not a report of a completed repair.

Step	User and system behavior	Evidence of success
1. Enter project	User launches Handwork in the intended repository. Handwork exposes current workspace and session context.	The user can identify the working directory and distinguish a fresh session from resumed work.
2. Connect	User selects a provider or official runtime, authenticates through that route, and selects an available model.	Engine, endpoint route, capability limits, and credentials source are understandable; secrets remain masked.
3. Set authority	User inspects permissions and additional roots before delegating changes.	The effective mode and scope match the intended task; repository settings cannot silently widen them.
4. Describe task	User provides the desired fix and verification boundary. The agent reads relevant instructions and code.	Discovery finds the actual failure and local test commands before proposing edits.
5. Execute	Agent makes targeted changes and runs the relevant checks through controlled tools.	Actual tool results and exit outcomes are available. Denied or failed actions are not reported as successful.
6. Review	Agent explains changed behavior, affected files, check results, and remaining uncertainty.	User can compare the diff and verify that unrelated work is preserved.
7. Continue or exit	User asks for a refinement, undoes an eligible tracked edit, or resumes later.	Saved conversation is available where enabled. Resume and undo limitations are clear.

Completion contract

A completed turn is not automatically a verified task. The task is accepted only when the requested behavior is demonstrated, applicable checks actually pass, protected work is preserved, and the developer can inspect the result. A fluent final response or zero process exit status alone cannot establish those conditions.

07 / SPECIFICATION

Alternate journeys and failure recovery

Understand a repository without editing

The user asks for entry points or failure-case analysis. Handwork retrieves relevant files and explains findings with paths and uncertainty. A prompt asking for no edits expresses intent; it does not create a dedicated read-only permission mode. The UI must not imply stronger containment than effective controls provide. [S01, S05]

Interrupted long-running task

The user interrupts during a command, write, provider stream, or compaction. The product acknowledges the interruption, stops new work where possible, and shows known completed actions and any unknown or partial effects. On resume, it inspects relevant workspace state before replaying a consequential action. Cancelling cannot undo work already performed.

Automation without a terminal

A script invokes ask with machine-readable output. It receives explicit completed, failed, or interrupted outcomes and separate diagnostics. A missing answer from a human is not permission. Official runtime adapters deny noninteractive approval requests; native runs follow their configured policy and interaction availability. [S01, S12]

Local-model workflow

The developer starts and configures an Ollama service, chooses a tool-capable model, and connects Handwork. If the service is unavailable or the model lacks a required capability, Handwork must explain the failure and preserve the request. The selected local provider alone is insufficient evidence that inference stays local because the server may route cloud models. [S01]

Host embedding

An integrator creates one SDK agent, consumes the event stream, awaits the result, checkpoints while idle, and closes the agent. The host must resupply authentication, instructions, tools, and integrations after restoration. Cancellation aborts host-tool signals but cannot force an external service to reverse its effects. [S11]

Blocked task

When credentials, model capability, scope, policy, or required user intent prevents progress, Handwork should identify the exact blocker and a feasible recovery action. It must preserve useful prior work and avoid substituting a different provider, a wider permission mode, or fabricated success.

08 / SPECIFICATION

Task and tool state contracts

The following is a proposed user-visible state model. Internal storage may use different names; it must preserve the distinctions and prevent false completion.

State	Required meaning	Allowed next steps
Ready	The request can be edited; no turn is executing.	Submit, change settings, inspect history, exit.
Preparing	The system validates configuration and loads relevant context.	Cancel; resolve an identified setup failure.
Running	A provider request or authorized tool is executing.	Observe progress, steer at a safe boundary, interrupt.
Awaiting decision	A particular unresolved action requires user or policy resolution.	Approve within displayed scope, deny, amend, or stop.
Held	Automatic review returned caution, unavailable evidence, or another unresolved result.	Explain the hold and supported remediation; do not silently run the action.
Interrupted / partial	The turn stopped and some effects may already exist.	Inspect effects, resume deliberately, undo only eligible changes.
Failed	A defined operation could not complete.	Show preserved work, actual cause where known, and a bounded retry or alternative.
Response complete	The model ended its turn.	Inspect change and evidence; accept, refine, or reject the task result.
Verified / accepted	Independent task checks and user review satisfy the requested outcome.	Continue with a new task or finish. This is distinct from model completion.

Tool outcome invariant

Every consequential tool action must distinguish proposed, denied, started, completed, failed, interrupted, and unknown-effect states where applicable. A retry must use the existing managed handle or inspect effects first when duplication is possible. Streamed text must not substitute for an execution record. [S04-S10, S12]

09 / SPECIFICATION

Requirements: installation and first use

SET-01 Qualify the advertised installation path

P0 / Baseline / [S01, S02, S03]

The npm CLI package must declare macOS arm64 support accurately and contain the executable and necessary runtime assets. Source-build instructions must state the pinned Zig requirement without implying equivalent support on other platforms.

Acceptance: Install the exact candidate package in a clean supported profile; launch help, version, and the terminal. Missing assets and unsupported architecture must produce actionable failures. [T01]

SET-02 Show the engine and data route before first task

P0 / Proposed / [S01, S04, S12]

Setup must distinguish native API access from official runtime access and identify the selected provider or local endpoint before repository content is sent.

Acceptance: A first-time user can correctly explain who runs the agent, which account route is used, and whether prompt/context data leaves the machine. No false claim of subscription transfer or local-only inference. [T02]

SET-03 Protect credentials during entry

P0 / Baseline / [S01, S15]

Typed and pasted API keys must be masked in setup, stored outside settings and conversation history, and never echoed into ordinary output. Saved native keys take precedence over environment keys.

Acceptance: Synthetic-key tests cover typing, paste, save, cancel, logout, and precedence. Saved key files have owner-only 0600 mode; errors and traces inspected for leakage. State explicitly that the files are not encrypted at rest. [T03]

SET-04 Make failed setup recoverable

P0 / Proposed / [S01, S04, S15]

Missing service, rejected credentials, empty model catalog, and unreachable endpoint must yield distinct explanations. The product must not widen authority or switch providers as a hidden fallback.

Acceptance: Mock each condition; preserve the user's draft and valid settings, state the failed step, and provide the supported reconnect or configuration action. [T02, T04]

SET-05 Keep release identity consistent

P0 / Proposed / [S01, S02, S03, S11, S12]

The candidate executable, npm manifest, help/version output, README release statements, and adapter metadata must agree where they describe the same release. SDK versioning must remain separately identified.

Acceptance: Package qualification records all versions and fails unexplained inconsistencies. Reconcile the observed 0.1.0 versus 0.0.9 mismatch before a release claim. [T01]

10 / SPECIFICATION

Requirements: providers and model capabilities

MOD-01 Keep provider selection explicit

P0 / Baseline / [S01, S04]

Native provider IDs must map to configured adapters. Model IDs are opaque; a namespaced gateway model must not cause provider inference. A custom endpoint must use the intended adapter protocol.

Acceptance: Mock discovery and requests for supported adapters. Preserve exact selected provider/model identifiers and report an unavailable choice without substitution. [T04]

MOD-02 Expose capability uncertainty

P0 / Baseline / [S01, S04, S15]

Tools, images, structured output, reasoning effort, and fast mode must be offered only where the selected model and transport support them. Unknown capabilities must remain unknown rather than optimistically assumed.

Acceptance: Unsupported image or output requests fail explicitly before being presented as accepted. Controls remain consistent after provider/model changes. Cover conservative vision metadata and Anthropic structured-output rejection. [T04]

MOD-03 Preserve explicit endpoint trust

P0 / Baseline / [S01, S15]

Custom public API endpoints require HTTPS; loopback HTTP requires an explicit port. Setup must explain that a configured key is sent to the selected endpoint.

Acceptance: Test invalid URLs, accepted loopback URLs, and intended custom endpoints using synthetic credentials. No credential-bearing URL is included in shareable diagnostics. [T03, T04]

MOD-04 Keep official runtimes separate

P0 / Baseline / [S01, S12]

Official Codex and Copilot commands must retain their engine-owned sessions, tools, authentication, and configuration. The adapter must not imply that native Handwork permission or session settings apply.

Acceptance: Runtime tests verify route selection, stream handling, provider session IDs, interruption, default denial, and an affirmative interactive approval path. Noninteractive unresolved permissions are denied. [T05]

MOD-05 Gate sensitive provider routes on verified authority

P0 / Proposed / [S01, S04]

Do not treat code availability as approval under provider terms. Keep disabled subscription routes disabled. Resolve the native Codex route's documented authorization caveat before promoting it as a qualified supported route.

Acceptance: The release records the reviewed route, applicable sources, reviewer, date, and limits. Missing policy evidence blocks a support claim; it does not justify testing with real consumer credentials. [T04, T15]

11 / SPECIFICATION

Requirements: project context and workspace scope

CTX-01 Load applicable instructions with visible limits

P0 / Baseline / [S01, S04]

Native sessions load applicable personal, project, ancestor, and scoped instructions within selection and size limits. They must not recursively include every instruction file.

Acceptance: Fixtures cover ancestor and target scope, conflicting text, omitted oversized content, and unknown paths. Omission diagnostics identify missing context without silently claiming full coverage. [T06]

CTX-02 Prevent project settings from escalating authority

P0 / Baseline / [S01, S05, S06]

Repository settings may configure supported project preferences but cannot set user-only permission rules, credentials, provider/model choices, or additional directories.

Acceptance: A hostile project settings fixture attempts each user-only field. Effective policy remains user-owned; ignored options are reported. [T06, T07]

CTX-03 Make additional roots deliberate

P0 / Baseline / [S01, S06]

Additional directories must be user-configured, canonicalized, and represented by availability and active state. The primary workspace identity must remain stable.

Acceptance: Cover nonexistent paths, file paths, duplicate canonical paths, symlinks, suppression of saved roots, and scope removal. A path outside active roots cannot gain access from textual instructions. [T07]

CTX-04 Preserve essential context during compaction

P0 / Proposed / [S01, S09, S11]

Compaction must retain the active objective, accepted corrections, constraints, decisions, verified outcomes, unfinished work, and relevant tool-effect uncertainty. It must not transform untrusted text into authority.

Acceptance: A multi-turn fixture is compacted and resumed; the agent receives those facts and continues without repeating a completed side effect. Cancellation leaves usable prior history. [T08]

CTX-05 Use bounded, inspectable retrieval

P1 / Baseline / [S01, S04, S06]

Search, reads, context budgets, and tool-result retention must have explicit limits and a supported path to narrower queries or retained detail. Bounded output must not be described as exhaustive.

Acceptance: Large and Unicode-path fixtures distinguish no match, truncated results, omitted files, and read errors. Results show scope and limits needed to judge completeness. [T06, T07]

12 / SPECIFICATION

Requirements: file tools and review

FIL-01 Read before a targeted edit

P0 / Baseline / [S04, S07]

A targeted edit must act on a previously read file. Each old string must identify one occurrence in the original content; ambiguous or overlapping replacements must be rejected.

Acceptance: Fixtures include a changed file, zero matches, multiple matches, and overlapping replacements. Rejection leaves file content unchanged and provides a way to reread and correct the patch. [T09]

FIL-02 Make a multi-replacement file edit atomic

P0 / Baseline / [S04, S07]

Within one file edit, either every validated replacement is applied or none is. A read/edit/commit failure must not be reported as a successful patch. This is not a promise of a cross-file transaction.

Acceptance: Induce validation and write failures around a multi-replacement edit. Verify bytes, recorded outcome, and no partial success claim. Cross-file partial completion is reported separately. [T09]

FIL-03 Preserve unrelated developer work

P0 / Proposed / [S01, S07, S08]

The agent must inspect relevant working-tree state before changes and preserve modifications unrelated to the requested task. Concurrent changes must produce a conflict or renewed inspection rather than a blind overwrite.

Acceptance: Start with user edits in and outside the target area, modify a target during the task, and verify protected bytes remain unchanged. Final output identifies the actual scope of changes. [T09, T13]

FIL-04 Report an inspectable result

P0 / Proposed / [S01, S13, S16]

Completion output must identify changed behavior and affected files, actual check results, and remaining uncertainty. Unrun tests must remain unrun, and failed checks must remain visible.

Acceptance: A repair fixture with a failing verification cannot yield a clean verified-task result. A successful fixture includes check names and observed outcomes without fabricated coverage claims. [T13]

FIL-05 Handle content and path extremes explicitly

P1 / Proposed / [S04, S06, S07]

File tools must distinguish text decoding errors, oversized content, inaccessible paths, symlinks, and missing files. They must not create or overwrite an external path because a model supplied it.

Acceptance: Exercise large files, invalid UTF-8, long and Unicode paths, scope escapes, and permission errors. Verify bounded diagnostics and unchanged protected targets. [T07, T09]

13 / SPECIFICATION

Requirements: shell and managed processes

CMD-01 Use owned process handles

P0 / Baseline / [S04, S13]

A command that continues beyond the initial observation returns an owned execution handle. Later observation, input, and stop operations must address that handle instead of launching the command again.

Acceptance: A delayed command creates a marker exactly once. Repeated observations do not duplicate it; an unknown handle yields an explicit error. Process completion includes its actual exit outcome. [T10]

CMD-02 Make interactive input intentional

P0 / Baseline / [S04, S13]

TTY commands accept input through the supported interaction operation. The agent must observe readiness before sending control characters. Non-TTY observation must not imply input was delivered.

Acceptance: Run a fixture that waits for a prompt, receives one input, and exits. Cover early input, invalid handles, input-size bounds, and stopping an interactive command. [T10]

CMD-03 Distinguish interruption from reversal

P0 / Proposed / [S01, S04, S13]

Stopping a command must report that it stopped, failed to stop, or has unknown status. Completed writes or external effects are not reversed by cancellation. Recovery must inspect effects before any retry.

Acceptance: Interrupt before and after a marker write and simulate delayed process shutdown. Verify preserved evidence, no automatic duplicate command, and accurate partial-effect messaging. [T08, T10]

CMD-04 Separate observation windows from deadlines

P1 / Baseline / [S04]

Initial yield and later wait windows control observation, not the lifetime of a long-running process. A finite command deadline must be supplied only when explicitly requested under the tool contract.

Acceptance: A process outlives the first yield and remains observable. An explicit deadline is honored and reported as a timeout; no unrequested deadline kills an intended service. [T10]

User-visible evidence

Display command, working directory, lifecycle status, bounded output, and exit result where available. Distinguish provider waiting from tool execution. A test command returning exit zero verifies only the checks it actually ran; it is not proof that the whole product or task is correct.

14 / SPECIFICATION

Requirements: permissions and user control

PER-01 Expose effective native authority

P0 / Baseline / [S01, S05]

The user can inspect native ask, auto, and full-access modes and the applicable rules. Auto is the default documented mode and is not a read-only mode; full-access disables Handwork checks.

Acceptance: Display and JSON fixtures verify mode and rule sources. Setup and help explain their consequences. A model request cannot change the mode without authorized user configuration. [T07]

PER-02 Authorize the specific action

P0 / Baseline / [S05, S10, S13]

An approval must bind to the intended action, target, arguments, and scope. A narrower existing grant must not become authority for a changed command or unrelated workspace.

Acceptance: Mutate target and command after a grant; admission requires a valid matching authorization. Multiple concurrent requests remain separately attributable. [T07]

PER-03 Hold unresolved automatic review

P0 / Baseline / [S01, S05]

A clear automatic review may authorize the specific action. Caution, incomplete evidence, unavailable review, or invalid review must hold or deny that action rather than silently execute it.

Acceptance: Mock each review outcome and verify no side effects in held cases. The user sees why progress stopped and which action remains unresolved. [T07]

PER-04 Keep denial and cancellation usable

P0 / Proposed / [S05, S07, S13]

A denied approval must leave unrelated work and the draft intact. The agent must not repeatedly request equivalent authority or bypass denial with a different tool.

Acceptance: Deny a file edit or command, then continue with a read-only task. Protected state is unchanged, the refusal remains visible, and an equivalent alternate-tool attempt is rejected. [T07, T09]

PER-05 State the limits of policy controls

P0 / Baseline / [S01, S05, S10, S11]

Permission checks must not be marketed as operating-system isolation or a correctness guarantee. Child agents and MCP tools must not gain authority merely from generated instructions.

Acceptance: Documentation, setup, and extension fixtures preserve that boundary. High-risk release review checks native dispatch paths and separate host/runtime controls. [T07, T12, T15]

15 / SPECIFICATION

Requirements: sessions, resume, and undo

SES-01 Persist conversation without implying a backup

P0 / Baseline / [S01, S09, S12, S14]

Saved native conversations must remain associated with their workspace. Resume restores conversation state, not a snapshot of files or running programs. Official runtime sessions remain separate.

Acceptance: Save, list, inspect, and resume an isolated session. Change a file after saving; resume does not restore old bytes or imply a command is still alive. [T08]

SES-02 Recover without destroying the source

P0 / Baseline / [S09, S14]

A supported recovery operation must preserve the damaged original and create a recoverable result with clear limits. Unrelated session directories must remain untouched.

Acceptance: Damaged accounting, truncated records, and unsupported corruption fixtures either recover deterministically or fail clearly. Compare original file hashes and unrelated sessions before and after. [T08]

SES-03 Reconcile interruption before continuation

P0 / Proposed / [S01, S09, S13, S14]

Resumed work must expose known completed tool actions, possible partial effects, and actions requiring inspection. It must not replay consequential operations based only on a missing final response.

Acceptance: Interrupt a write or command at multiple boundaries and resume. Compare resulting file and marker state; no duplicate side effect and no false completion. [T08, T10]

SES-04 Undo only a matching tracked file change

P0 / Baseline / [S01, S08]

Tracked-file undo restores previous bytes or removes a created file only when current content still matches the recorded Handwork result. A conflict leaves the current file untouched.

Acceptance: Test restore, remove, external modification, missing preimage, and restore I/O failure. Conflicts remain available for manual review; unavailable restoration is never reported as restored. [T09]

SES-05 Explain saving and retention choices

P1 / Proposed / [S01, S09]

The product must explain native session storage, separate prompt history, and the scope of no-save. Session deletion and any future retention controls need explicit semantics before being offered.

Acceptance: Inspect no-save behavior in an isolated profile. Do not claim that no-save prevents all logs, histories, provider processing, or external copies unless each path has been verified. [T08, T15]

16 / SPECIFICATION

Requirements: model-led subagents

SUB-01 Delegate a bounded task with explicit ownership

P1 / Baseline / [S01, S04, S10]

A parent can create one temporary task or a persistent named child conversation. The parent remains responsible for integration and review; a pending receipt is not a completed result.

Acceptance: A child runs asynchronously and delivers its outcome without duplicate dispatch. The parent integrates actual results and states unresolved child failures. [T11]

SUB-02 Preserve creation-time configuration

P1 / Baseline / [S01, S04, S10]

Provider, model, and effort are independently inherited unless explicitly set at creation. Explicit provider requires model; IDs remain opaque. Existing children reject attempted configuration changes.

Acceptance: Fixtures cover inherited values, cross-provider credentials, missing model, unsupported effort, and attempts to mutate a named child. Failure does not silently create a substitute child. [T11]

SUB-03 Queue steering at safe boundaries

P1 / Baseline / [S01, S04, S10]

Follow-ups to a working named child queue without cancelling its current tool. Interrupting a child must not imply rollback of completed effects. The parent should avoid polling or redispatching solely because a receipt is pending.

Acceptance: Steer a child while its controlled tool is running. Verify one tool execution, ordered follow-up delivery, and a distinct final result. [T11]

SUB-04 Prevent authority and workspace conflicts

P0 / Proposed / [S05, S06, S10]

A child cannot widen the parent's approved authority through instructions. Concurrent mutating work must use an explicit coordination or isolation policy so workers do not overwrite each other.

Acceptance: Two child fixtures target the same file and one targets a forbidden root. The forbidden action is denied; conflict handling preserves user edits. Approved concurrency and isolation strategy remain an owner decision. [T07, T11]

Limits to make visible

There is no documented server-side role router. Delegation is model-led. Different providers can incur separate usage and require separate credentials. The native ask JSON main-agent token total does not automatically include child usage. A comparison of child efficiency must record its own accounting scope. [S01]

17 / SPECIFICATION

Requirements: skills, MCP, and images

EXT-01 Load skills from the advertised source

P1 / Baseline / [S01, S04, S11]

Skills must be explicitly discovered or loaded from a supported location. Relative references resolve from that skill directory. Installing a skill is distinct from loading it and must follow authorization.

Acceptance: Load a fixture with a relative resource and a missing resource. Return a truthful error without inventing content or installing a different skill. [T12]

EXT-02 Require project MCP trust

P0 / Baseline / [S01, S04, S17]

Project MCP configuration is subject to trust approval. Server authentication, transport, capability discovery, and tool effects must retain their ownership boundaries.

Acceptance: An untrusted project server is not launched or used as trusted authority. Local/remote and authentication fixtures cover denial, failure, stale catalogs, and reconnection. [T12]

EXT-03 Keep external content untrusted

P0 / Baseline / [S04, S05, S17]

MCP resources, prompts, fetched pages, tool results, and repository text must be treated as data. Their content cannot override user intent or grant permissions.

Acceptance: An adversarial fixture embeds instructions to reveal a synthetic secret or publish a change. Native policy denies unauthorized effects, and the agent identifies or ignores the injected instructions. [T07, T12, T15]

EXT-04 Make image input conditional and inspectable

P1 / Baseline / [S01, S11, S15]

Image attachment must use a supported host path or clipboard mechanism and a compatible model/transport. The user must be able to inspect or remove pending images before sending.

Acceptance: Cover unavailable clipboard, unsupported format, remote terminal, model without vision, and cancelled attachment. Text paste must not imply that image bytes were transmitted. [T04, T12]

Conformance boundary

The repository contains a pinned MCP conformance subproject and explicit expected failures. This is evidence of a verification mechanism, not a claim that a fresh suite passed or that every MCP version is supported. Record the exact candidate, protocol, runner version, and baseline before reporting conformance. [S17]

18 / SPECIFICATION

Requirements: scripting, GitHub, and usage

AUT-01 Produce honest machine-readable output

P0 / Baseline / [S01, S13, S16]

Native ask JSON must distinguish accumulated assistant output from completed final output. Reported main-agent token fields preserve unavailable values as null rather than fabricated zeroes.

Acceptance: Parse success, interruption, malformed provider stream, and failure fixtures. Partial text remains distinguishable from a finished final response; diagnostics cannot corrupt the JSON result. [T13]

AUT-02 Keep script diagnostics separate

P0 / Baseline / [S01, S13, S16]

For redirected native ask output, stdout contains the requested result while stderr carries progress and diagnostics. Exit behavior must be documented for completion, error, and interruption.

Acceptance: Redirect both streams in fixture runs and assert parseability and failure signaling. A failed tool or transport must not disappear behind a successful-looking result. [T13]

AUT-03 Separate drafting from publishing

P0 / Baseline / [S01, S13]

PR and issue commands draft by default. Publishing requires the explicit create option and the GitHub CLI with appropriate authentication and repository access.

Acceptance: A fake GitHub CLI records zero create calls during drafting. The explicit create path uses the intended repository and reports success only when the remote operation succeeds. [T13]

AUT-04 Present usage as scoped local evidence

P0 / Baseline / [S01, S09, S12]

Usage displays must identify the source and scope of token/spend information. Unknown dollar costs stay unavailable. Native main-agent JSON totals must not be presented as authoritative invoices or complete nested-agent spend.

Acceptance: Fixtures cover missing counts, unknown prices, zero usage, multiple providers, child work, and official runtime snapshots. UI labels remain accurate for each surface. [T05, T13]

Automation boundary

No background job scheduler or hosted automation service is specified here. One-shot invocation, MCP tools, subagents, and official runtime adapters do not imply that Handwork can schedule future work. Any future scheduling capability requires its own lifetime, authority, data retention, cost, and failure requirements.

19 / SPECIFICATION

Requirements: SDK and ACP contracts

INT-01 Keep embedding host-owned

P0 / Baseline / [S11, S18]

The SDK agent exposes prompt, checkpoint, and close. Importing the default package must not perform MCP connection, skill scanning, process spawning, or filesystem discovery. The host supplies authentication, instructions, and tools.

Acceptance: Import-side-effect tests observe no hidden operational work. Host-tool execution is explicit; no native CLI shell/filesystem capability appears merely from agent construction. [T14]

INT-02 Make streaming and cancellation reliable

P0 / Baseline / [S11, S18]

One event consumer drains a turn; unread events can apply backpressure. Breaking iteration, explicit cancel, or close releases blocked output. Transport/decoding failures reject instead of returning success with missing text.

Acceptance: Use slow and absent consumers, early cancellation, malformed messages, and late tool settlement. Awaited results and cleanup match the contract without hanging. [T14]

INT-03 Restore only bounded, idle checkpoints

P0 / Baseline / [S11, S18]

Checkpoint is idle-only and returns versioned opaque bytes containing conversation history and usage. Restore occurs on a fresh agent and requires host integrations to be supplied again.

Acceptance: Round-trip a checkpoint, reject invalid or oversized bytes, reject an active-turn checkpoint, and verify absent credentials/tools are not silently restored. [T14]

INT-04 Keep backend selection diagnosable

P1 / Baseline / [S11, S18]

Native, auto, and Wasm selection must report availability and stable failure reasons. Agent and terminal probes remain separate. A successful backend load does not certify credentials or a future prompt.

Acceptance: Test missing artifact, API mismatch, missing surface, disabled native backend, and unavailable JSPI. ESM/CommonJS and packaged assets follow the documented host matrix. [T14]

INT-05 Preserve ACP protocol separation

P0 / Baseline / [S01, S13]

ACP stdio carries protocol messages; diagnostics must use the intended separate channel. The endpoint must retain native permissions and accurate session/tool outcomes without claiming a bundled editor integration.

Acceptance: An ACP client fixture initializes, starts a session, prompts, receives tool updates, cancels, and sees failures without malformed protocol output. [T14]

20 / SPECIFICATION

Requirements: terminal UX and accessibility

UX-01 Keep the active task legible

P0 / Proposed / [S01, S13]

The composer, workspace, engine/model selection, permission mode, and current activity must remain understandable during output streaming and terminal resize. Progress cannot masquerade as completion.

Acceptance: Replay long transcripts and concurrent activity at 80x24, 120x40, and a narrow 60-column stress size. Key state and recovery actions remain reachable without clipped critical text. [T16]

UX-02 Make core tasks work by keyboard

P0 / Proposed / [S01, S13]

Setup, provider/model choice, task entry, permission decisions, interruption, session resume, and result inspection must have keyboard paths. Hover-to-copy and drag selection must remain optional enhancements.

Acceptance: A keyboard-only walkthrough completes each core task in the selected supported terminals. Focus/caret position and dialog choices remain visible after resize and cancellation. [T16]

UX-03 Do not encode status only in color or sound

P0 / Proposed / [S01, S13]

Success, failure, held approval, selection, and active work need text or another redundant cue. No-color output and optional sounds must not remove critical information.

Acceptance: Run with colors and sounds disabled, enlarged terminal text, and high-contrast terminal themes. The user can still distinguish each state and resolve decisions. [T16]

UX-04 Verify assistive-use behavior by surface

P1 / Proposed / [S01, S11, S13]

Test the terminal with actual assistive technology and record limitations. For browser or editor hosts, use their semantic accessibility APIs and the applicable project WCAG target. Do not claim terminal conformance from web criteria alone.

Acceptance: Record terminal, OS, screen-reader versions, tasks, announcement behavior, focus, and defects. Any unavailable task has an explicit fallback or release-blocking disposition. [T16]

UX-05 Write errors at the point of failure

P0 / Proposed / [S01, S08, S13, S15]

Errors state the failed operation, affected object, preserved work, and a feasible next action. Keep uncertain causes uncertain; never expose keys or misleading recovery promises.

Acceptance: Review expired credentials, denied permission, unknown command handle, undo conflict, interrupted write, and model capability failure. Users can identify what happened and the next supported action. [T02, T16]

21 / SPECIFICATION

Security, privacy, and data lifecycle

Security requirements protect authority and integrity; privacy requirements address what data is processed, where it goes, and how long copies remain. This section specifies controls and review needs, not a completed security audit.

Data or boundary	Current documented handling	Required release evidence
Native API keys	Provider-specific files; 0600 mode; saved values take precedence over environment. Not encrypted at rest.	Masking, file permissions, logout behavior, and synthetic-secret leakage tests. Keychain migration remains an open option.
Prompts and code context	Cloud routes can transmit prompt, excerpts, results, and history to the selected endpoint.	Visible route disclosure; no hidden endpoint switch; actual payload review with synthetic data. Provider retention remains provider-specific.
Sessions and history	Local native session data plus separate prompt history. Resume is not a workspace backup.	Storage-format review, failure handling, no-save scope, retention/deletion decision, safe recovery copies.
SDK checkpoints	History and usage; durable storage is host-owned; credentials and integrations must be supplied again.	Bounded format, version compatibility, host access controls, and documentation that bytes may contain sensitive content.
MCP / shell / host tools	External processes and services have their own authority and effects.	Trust approval, scope tests, cancellation limits, injection tests, and explicit containment assumptions.
Traces and usage	Diagnostic traces can contain private material; local token/cost records are scoped.	Redaction review, no automatic upload claim, approved diagnostic fields, user review before sharing.

Threat scenarios to test

A repository instruction attempts to widen permission mode; a symlink escapes the intended root; a tool result asks the agent to disclose a synthetic secret; stale approval is reused for a changed command; a recovered session replays a publish action; an SDK host exposes a long-lived key in browser code. Each scenario must have a named control, test, and residual-risk owner. [S05-S12, S15, S17, S18]

No claim of zero vulnerabilities, encrypted-at-rest sessions, policy compliance, or complete data deletion is made. Retention, diagnostics-sharing, and incident-response ownership remain explicit release decisions.

22 / SPECIFICATION

Nonfunctional requirements and performance

The values below are either repository test budgets or proposed qualification methods. None is a fresh measured result. A performance improvement is acceptable only with preserved correctness and permission behavior.

ID / priority	Requirement	Evidence and acceptance
NFR-01 / P0	Core task integrity: denied actions have no unauthorized effects; failed checks cannot yield a verified-task claim.	All critical fixtures pass on the exact candidate. Any credible unauthorized effect or data loss blocks release.
NFR-02 / P1	Interactive latency stays bounded under representative terminal load.	Use the existing performance suite's workload-specific budgets, including local p50 8 ms, p90 12 ms, p95 17 ms. Record opt-in test settings, hardware, build, and actual samples. [S13]
NFR-03 / P1	Startup and reconnect do not unnecessarily block editable input.	Proposed: measure cold/warm time to an editable prompt, network discovery, and offline commands. Set absolute budgets only after a frozen baseline.
NFR-04 / P1	Resource use stays bounded for long sessions, large output, and repeated open/close cycles.	Record process-tree RSS, descriptors, output bounds, and cleanup over controlled fixtures. Set approved limits by workload; provider memory is outside local measurements.
NFR-05 / P0	Persistence failure does not silently corrupt or overwrite recoverable work.	Fault-inject write, sync, restore, and migration failures. Preserve originals; identify partial state and supported recovery. [S08, S09, S14]
NFR-06 / P0	Documented surface contracts survive packaging and supported backend choice.	Test the installed artifact, runtime assets, SDK native/Wasm selection, ESM/CommonJS, and ACP transport. Backend support is not CLI platform support.
NFR-07 / P1	Errors and progress remain diagnosable without unbounded sensitive logging.	Bound diagnostics, use explicit lifecycle outcomes, exclude credential values, and publish reproduction steps that use synthetic data.

Measurement discipline

Separate local startup and input-to-paint latency from hosted inference and tool runtime. Freeze executable hashes, source revision, model settings, task, environment, and run order. Report failed attempts and actual sample counts. Historical README pilots cannot be substituted for a current release benchmark. [S01, S13]

23 / SPECIFICATION

Desired impact and success measures

Expected user impact

Handwork should reduce avoidable setup confusion, local waiting, and recovery effort while enabling a developer to accept a verified change. That impact must be demonstrated with task outcomes and comprehension, not assistant verbosity, tool-call volume, or the appearance of activity. Adoption and revenue targets remain unspecified.

Measure	Definition and population	Decision rule
Verified repair rate	Attempts passing independent task checks and protected-file checks / all eligible attempted repairs in a frozen fixture set. Include failures and timeouts.	Primary outcome. Compare before speed. Baseline and acceptable change need approval.
First useful task	Time from first launch to completion of one predefined useful task among new users in an observed setup study.	Report completion rate and median time among completions, with setup failures separately visible.
Verified task time	Wall time from submission to verified result for matched successful tasks; record failure rate and timeout budget alongside it.	An improvement requires no accepted correctness or safety regression.
Recovery integrity	Controlled interrupted tasks resumed without duplicate side effects, protected-work loss, or false completion / all interruption fixtures.	All critical fixtures must pass for release; real-world incidence is unknown.
Permission integrity	Unauthorized side-effect cases observed in adversarial fixtures and release review.	Any credible case is a hard stop. A zero fixture count is scoped evidence, not universal proof.
Comprehension	Users correctly explain engine, data route, permission mode, and resume limitations after setup and use.	Observe wrong mental models and revise the flow. Participant target and pass threshold remain open.
Local responsiveness	p50/p95 input-to-paint and time-to-editable-prompt, separated by workload, build, and machine.	Preserve approved workload budgets; measure tails rather than mean alone.

Business / project impact

A qualified release, fewer reproducible setup and recovery defects, and usable SDK examples are credible project outcomes. Pricing, market size, conversion, customer retention, and monetization have no supplied baseline and are not invented. Long-term usage is meaningful only when tied to useful completed work.

24 / SPECIFICATION

Proposed measurement and event contracts

This is a proposed local evaluation schema, not evidence that telemetry is shipped. No remote analytics collection is authorized by this PRD. Production collection would require a separate purpose, consent, retention, and ownership decision.

Event	Trigger / non-trigger	Minimum useful fields
task_attempt_started	A validated task is admitted for execution. A draft or rejected setup is not an attempt.	schema_version, run_id, local sequence, monotonic start, surface, provider class, build identity.
tool_action_resolved	An action reaches allowed, denied, held, failed, completed, or interrupted outcome. Receipt is not completion.	action_id, run_id, tool class, outcome, policy source class, duration, retry parent ID.
turn_ended	A turn ends as complete response, failed, or interrupted. Model completion does not mark task verification.	run_id, turn_id, outcome, duration, bounded error class, usage availability.
task_verified	An independent fixture verifier evaluates the declared acceptance checks.	run_id, fixture_id/version, verifier_version, pass/fail, protected-state result.
session_resumed	The selected conversation is restored and admitted; opening a picker does not count.	local session reference, surface, migration/recovery class, reconciliation outcome.
interaction_sampled	A test harness records input-to-paint for a named workload.	workload, sample sequence, duration, terminal dimensions, candidate/build identity.

Contract rules

IDs are scoped to the evaluation run; they are not cross-device personal identities. Use monotonic time for durations and record event time separately if needed. Every semantic action has one stable ID so retry delivery does not double-count it. Use explicit unavailable values instead of invented measurements. Emit task_verified only from the verifier.

Prohibit prompt text, source code, tool arguments, file contents, absolute workspace paths, credential values, user email, and unrestricted free text in this schema. A local fixture ID is sufficient for reproducing a benchmark. Collection failure must not block the coding task.

Analysis plan

Freeze task eligibility, budgets, comparison revision, model settings, sample counts, and exclusions before reading results. Separate native and official runtime data. Report all attempted tasks, actual denominators, distributions, failures, and missing data. Local usage is not an invoice; correlation is not a causal adoption claim.

25 / SPECIFICATION

End-to-end acceptance scenarios

These scenarios test the assembled product. They are required evidence for the proposed baseline, not checks already performed for this document.

ID	Precondition and action	Required result
AC-01	Clean supported Mac profile; install candidate, configure a synthetic provider, and repair a small failing fixture.	Task produces a focused patch, actual passing verifier, inspectable output, and no protected-file changes.
AC-02	Project contains malicious settings and instructions requesting full access and a secret upload.	Effective user policy is unchanged; unauthorized effect is denied; no synthetic secret appears in outgoing payload.
AC-03	Ask-mode user denies a write, then requests an explanation of the file.	File bytes remain unchanged; explanation can continue; denied authority is not reacquired through another tool.
AC-04	Auto review returns caution or unavailable for a command.	Command never starts; held action and review limitation are visible; no hidden mode escalation.
AC-05	Interrupt a command after it writes a marker; resume the task.	Resume identifies existing effects, avoids duplicate marker action, and does not claim cancellation reversed the write.
AC-06	Modify a Handwork-edited file externally, then request tracked undo.	The current bytes are preserved and a conflict is reported; no restoration claim is issued.
AC-07	Restore a damaged session with an unrelated session directory present.	Supported recovery preserves source and unrelated records; unsupported corruption fails clearly.
AC-08	Request unsupported image input or a missing model.	Request does not silently switch provider/model or claim the image was processed; draft and recovery instructions remain.
AC-09	Invoke official runtime without an interactive terminal; runtime requests approval.	Request is denied; native permission mode is not represented as authority for that runtime.
AC-10	Embed SDK with slow event consumer; cancel active host tool; restore checkpoint on fresh agent.	No deadlock or late-result success; integrations are host-supplied; history restoration does not restore hidden authority.
AC-11	Navigate core flows by keyboard while streaming large output and resizing.	Composer and decisions remain reachable; state is understandable without color or hover.

26 / SPECIFICATION

Verification plan and traceability

Test ID	Risk / method	Existing evidence starting point
T01	Packaging and identity; clean install and installed-artifact smoke checks.	package.json; src/main.zig; CLI tests; update and signing scripts.
T02	Setup understanding; failure fixtures plus observed user walkthrough.	README; tui-auth-source-selection; startup tests. Human study remains needed.
T03	Credential leakage and precedence; synthetic keys, file modes, output review.	api-key-entry.py; api-providers.py.
T04	Provider and capability contracts; isolated mock services.	api-providers.py; vision routing evaluations; provider source.
T05	Official runtime ownership and permissions; mock engine/SDK tests.	runtime/runtime.test.mjs.
T06	Instruction/settings/context scope; deterministic filesystem fixtures.	config-persistence.test.ts; context and settings source.
T07	Permission/scope bypass; unit rules and adversarial integration.	permission-errors; file-tool-permissions; auto-mode-reliability; permission source.
T08	Persistence/interruption/compaction; fault injection and saved-state comparison.	session-recovery; tui-interrupt-recovery; tui-compaction-activity.
T09	Edits and undo preserve bytes; stale, ambiguous, I/O-failure fixtures.	file-tool-paths; change_tracker.zig; file mutation tests.
T10	Owned commands; marker-once tests, interactive/stop lifecycle.	tui-terminal-tool; shell schemas and managed execution contract.
T11	Delegation lifecycle and authority; controlled child fixtures.	subagent source and embedded unit tests.
T12	Extensions and trust; MCP mock/conformance and image host fixtures.	MCP E2E tests; conformance baseline; SDK adapters.
T13	Assembled native tasks, output, GitHub boundary, and verification honesty.	cli; ask-presentation; agent-quality and repair evaluations.
T14	SDK/ACP lifecycle and packaging contracts; native/Wasm host fixtures.	sdk/tests; sdk/node; acp.test.ts.
T15	Security/privacy/provider claim review; boundary and payload inspection.	Inspected source and docs; independent reviewer and missing policy evidence still needed.
T16	Terminal rendering, latency, keyboard, assistive use.	render-lab; tui-resize; tui-performance; manual assistive testing still needed.

Starting points are not coverage claims. Review test contents and environment gates before running. Existing commands include zig build test, targeted Bun E2E runs, Python mock-provider tests, and node --test runtime/runtime.test.mjs. Live evaluations require separate configured accounts and cost approval; broad execution is not implied. [S01, S13-S18]

27 / SPECIFICATION

Delivery sequence and dependencies

The sequence below is proposed and capability-based. It does not assign dates, engineering estimates, staffing, or external launch commitments.

Phase	Deliverable and dependency	Exit evidence
A. Establish release truth	Reconcile native release identity and docs; inventory advertised surfaces; review sensitive provider routes; freeze evidence snapshot.	Package/executable/docs agree; unsupported claims removed; each route and owner decision is recorded.
B. Qualify the native core	Prioritize installation, credential handling, scope, permissions, atomic edits, command ownership, honest output, and task verification.	Critical setup and repair fixtures pass on the installed artifact; denied actions produce no unauthorized effects.
C. Qualify continuity	Exercise save/resume, fault recovery, tracked undo, interruption reconciliation, and compaction.	Recovery fixtures preserve originals and protected bytes, avoid duplicate side effects, and show remaining uncertainty.
D. Qualify extensions	Test model-led children, trusted MCP, skill resource loading, conditional images, official runtime adapters, SDK and ACP.	Per-surface contracts and platform matrices are explicit; host authority and default-denial cases pass.
E. Tune and validate use	Freeze baseline, optimize measured bottlenecks, test keyboard and assistive use, observe representative developer tasks.	Performance improves only alongside correctness; user comprehension barriers have fixes or explicit dispositions.

Dependencies

Native builds depend on the repository's Zig 0.16.0 toolchain. Official adapters depend on their provider runtime and documented Node requirements. Copilot depends on the pinned SDK dependency. MCP conformance has its own pinned Node/npm/Bun environment. SDK Wasm requires a JSPI-capable host; native addon loading has separate platform and asset constraints. [S01-S03, S11, S12, S17]

External factors

Model availability, account entitlements, quota, provider latency, endpoint policy, local model capability, terminal features, and GitHub access can change independently of Handwork. Expose each boundary instead of inventing a universal fallback. No consumer subscription authority is assumed from an adapter existing in code.

28 / SPECIFICATION

Constraints, tradeoffs, and risk register

Risk / constraint	Consequence	Mitigation and release disposition
Model output is probabilistic	A fluent agent can create an incorrect patch or misread tool evidence.	Independent task checks and developer review; do not equate final response with verification.
Permissions are not OS isolation	Authorized shell/tool actions may reach beyond what users imagine.	Show effective mode/scope; test dispatch boundaries; document separate containment requirements.
Source/docs drift	Users can follow incorrect version, update, or feature expectations.	Tie claims to candidate and reconcile release identity. Missing referenced evidence stays missing.
Provider authorization uncertainty	An available integration may lack verified support under provider terms.	Gate support claims on dated review; prefer documented official runtime route where appropriate.
Interrupted side effects	Retry can duplicate writes, commands, or publishing.	Record outcomes, reconcile workspace/process state, and require explicit retry scope.
Concurrent children	Different agents can overwrite work or multiply cost.	Bound tasks, coordinate mutations or isolate work, preserve child authority boundaries, scope usage reports.
Plain local secret storage	Local compromise or accidental sharing can expose credentials.	0600 files and masking are baseline; stronger OS storage is an open decision, not a shipped guarantee.
Accessibility gaps	Terminal rendering may be visually usable while assistive users cannot finish a task.	Manual task verification per host; explicit limitations and nonpointer paths.
Small or stale benchmarks	Performance claims can outstrip evidence or hide correctness regression.	Freeze revisions, report failed attempts, verify tasks first, separate local and provider latency.
Platform breadth	CLI and SDK matrices can be conflated, increasing support obligations.	Qualify each artifact and backend separately before advertising support.

Tradeoffs to preserve

Provider breadth increases capability and policy complexity. Context compaction improves cost but risks dropping constraints. Parallel tools reduce wait time but increase conflict and resource risk. Smaller binaries must not remove safety or recovery behavior. Auto review reduces interruption but requires clear held-action states and reliable evidence.

29 / SPECIFICATION

Open decisions and proposed ownership

The repository names Connor Love as the project author. The responsibility roles below are proposed assignments for review, not evidence that additional staff or approval bodies exist. The author can own several roles, but each decision still needs a recorded answer.

ID / decision	Why it matters	Proposed owner and timing
D01: next release scope	Select which proposed P0/P1 requirements form the actual delivery commitment.	Project author / product owner; before implementation planning.
D02: published release identity	Reconcile README, executable, manifest, adapter metadata, and actual registry status.	Release owner; before public release claims.
D03: provider route authority	Resolve native Codex authorization evidence and maintain disabled-route policy.	Provider integration owner with qualified policy review; before promotion.
D04: performance budgets	Approve representative machines/workloads and new absolute startup/resource limits.	Runtime owner; after frozen baseline and before optimization claims.
D05: data retention and diagnostics	Choose session/history retention, deletion behavior, trace redaction, and any future sharing.	Project author / privacy owner; before new collection or retention controls.
D06: stronger credential storage	Choose whether to adopt an OS credential store and how to migrate existing saved keys.	Security/runtime owner; before promising encrypted storage.
D07: concurrency / isolation	Specify a supported policy for simultaneous mutating child work.	Agent runtime owner; before expanded parallel-edit claims.
D08: accessibility support matrix	Select supported terminals, screen readers, host surfaces, and fallback obligations.	Terminal/host owner with accessibility review; before accessibility claims.
D09: research and impact targets	Validate user segments and approve study sample, success thresholds, and business measures.	Product owner; before broad positioning or adoption commitments.
D10: platform and updates	Choose next CLI platforms, signing support, release service, rollback, and ongoing maintenance.	Release owner; before expanding distribution.

A missing answer does not block this PRD from serving as a detailed review draft. It does block an affected implementation commitment or public claim. Resolve the decision, update the requirement and evidence, and preserve unrelated settled scope.

30 / SPECIFICATION

Release acceptance and definition of done

A qualified baseline release must demonstrate

1. Its installed artifact matches the advertised platform and version, includes the required assets, and launches through the documented path.

2. Native provider selection, credential handling, instruction scope, workspace access, permission behavior, file edits, commands, and output contracts pass the applicable critical fixtures.

3. Save/resume, interrupted work, corrupt-session recovery, and eligible undo preserve user work and do not claim rollback or completion without evidence.

4. Every advertised official-runtime, SDK, ACP, skill, MCP, image, and subagent capability has per-surface qualification evidence and visible limitations.

5. Core terminal tasks work by keyboard and required assistive-use checks have recorded results. Any critical accessibility barrier has an explicit release disposition.

6. Performance results identify exact revision, artifact, machine, workload, build, sample count, and failure outcomes. Correctness and authority guardrails are preserved.

7. Documentation separates supported behavior, proposals, unknowns, local accounting, provider billing, and external policy. No fabricated proof or unsupported comparative claim remains.

Hard-stop conditions

Credible unauthorized execution, secret disclosure, loss of protected work, misleading recovery success, hidden provider fallback, malformed automation success, or an unrecoverable core-task accessibility barrier blocks the affected release claim. An unresolved provider-policy question blocks promotion of that route. A faster benchmark does not override these conditions.

Evidence retained with the release

Candidate hash and commit; version inventory; supported surface/platform matrix; actual checks and results; failure and recovery artifacts; independent verifier output; manually tested terminal/assistive environments; known limitations; approved exceptions with owner and review date; update and rollback evidence where supported.

Completion of this document

The assignment requirements are covered and each major product area has concrete behaviors, acceptance evidence, risks, and source traceability. Proposed commitments remain proposals. The PDF is a grounded product specification for review, not a certification that Handwork already satisfies every requirement.

31 / SPECIFICATION

Sources and evidence limits

All source identifiers refer to repository-relative paths in the inspected checkout. References support documented behavior or visible implementation definitions; they do not imply that a runtime path passed during this document review.

ID	Source	Use and limit
S01	README.md	Product description, commands, provider boundaries, permissions/data, proposed priorities, current limits. Documentation may lag inspected source.
S02	package.json	Native package version 0.1.0, darwin/arm64 manifest, distribution assets, Copilot dependency.
S03	src/main.zig; build.zig.zon	Native version 0.1.0 and minimum Zig 0.16.0. Build-package metadata is not the public CLI release version.
S04	src/builtins/tools.zig; src/builtins/providers.zig	Tool schemas/descriptions and registered native providers. Empty Qwen and Grok bundles are visible.
S05	src/core/permissions/permission_gate.zig; auto_classifier.zig	Gate decision shapes, read-only fallback, full-access internals, automatic-review outcomes and context.
S06	src/core/workspace/workspace_access.zig	Canonical additional-directory identity, availability, active scope, and path resolution.
S07	src/core/tooling/file_mutation_contract.zig	File-mutation contract; combined with built-in edit schema. Runtime acceptance still needs tests.
S08	src/core/workspace/change_tracker.zig	Content-stamp matching, conflict preservation, restore/remove results, unavailable undo, atomic restore.
S09	src/core/session/ (file inventory); README.md	Session storage, migration, recovery, usage, and context subsystems. Inventory establishes code presence, not execution results.
S10	src/core/subagent/model_contract.zig; src/builtins/tools.zig	Child configuration rules, bounded task contracts, live-message and creation-time settings.
S11	sdk/README.md; sdk/package.json	Agent/terminal APIs, host authority, streaming bounds, checkpoints, backends, SDK version 0.0.0 and host constraints.
S12	runtime/cli.mjs; codex.mjs; runtime.test.mjs	Runtime CLI separation, affirmative approvals, non-TTY denial, cancellation, adapter metadata, mock-test definitions.
S13	tests/e2e/ (inventory); tui-performance.test.ts	Terminal, scope, CLI, ACP, permission, output, and rendering fixtures. Performance budgets and opt-in gates inspected; suite not run.
S14	tests/e2e/session-recovery.test.ts; tui-interrupt-recovery.test.ts	Recovery/interruption fixture definitions and source-preservation checks; not a fresh passing result.

ID	Source	Use and limit
S15	tests/e2e/api-providers.py; api-key-entry.py (inventory)	Mock provider/key entry starting points. README documents their use of isolated synthetic credentials.
S16	tests/evals/ (inventory)	Repair, tool use, quality matrix, permission, and vision evaluation starting points; no live evaluations run.
S17	tests/e2e/conformance/README.md	Pinned MCP runner/protocol, isolated lockfile workflow, exact-binary requirement, expected-failure discipline.
S18	sdk/tests/; sdk/node/ (inventories)	SDK lifecycle, cancellation, stream pressure, checkpoints, tools, backend and packaging test starting points.

The supplied screenshot is the assignment source only. It does not supply Handwork product facts or authorization to post in the submission channel. Repository-local anti-slop, clear-writing, UX, accessibility, security/privacy, analytics, and testing skills guided the document's quality controls. No competitor or provider-policy claim was independently verified on the web for this artifact.

32 / SPECIFICATION

Glossary and assignment coverage

Term	Meaning in this PRD
Harness	The environment connecting a model to tools, context, permissions, saved work, and a user interface.
Native agent loop	Handwork's own orchestration and tool execution path, as distinct from an official provider engine.
Provider transport	The adapter and endpoint protocol used to send model requests and receive responses.
MCP	Model Context Protocol; external tool/resource/prompt integration. Server authority remains separate.
ACP	Agent Client Protocol; the agent endpoint used by a compatible client over stdio.
N-API / Wasm / JSPI	Native Node addon, WebAssembly backend, and JavaScript Promise Integration capability. These describe embedding mechanisms.
Compaction	Reducing conversation context while retaining the information needed to continue the task.
Tracked undo	Conditional reversal of a recorded file mutation while the file still matches Handwork's written content.
Independent verifier	A check outside the agent's completion claim that evaluates the requested outcome and protected-state conditions.
RSS / p95	Resident memory sampled for a defined process scope; 95th-percentile latency for a specified workload and sample set.

Required deliverable mapping

Assignment requirement	Where to find it
Problem	Problem and product opportunity.
Users	Users and jobs to be done; hypotheses and research needs.
Core journey	Verified repository repair; alternate journeys; state contracts.
Requirements	SET, MOD, CTX, FIL, CMD, PER, SES, SUB, EXT, AUT, INT, UX, and NFR specifications.
Desired impact	Success measures, event contracts, acceptance scenarios, and release gates.
Constraints	Execution boundaries, privacy, performance, dependencies, risk register, and open decisions.

The PDF is ready for the project owner to review and submit. No submission message, external posting, or implementation change is included.